

AgriConnect - Farm Link Intelligence

A comprehensive web application connecting farmers and buyers in Kenya, featuring crop listings with photos, real-time messaging, AI-powered agricultural planning, and integrated payment systems.

Table of Contents

- Introduction
- Features
- Technology Stack
- Installation
- Usage
- API Documentation
- Database Schema
- Testing
- Contributing
- License

Introduction

AgriConnect is a modern web platform designed to bridge the gap between small-holder farmers and buyers across Africa, particularly in Kenya. The application provides a marketplace for agricultural products with advanced features like AI-powered crop planning, real-time communication, and role-based dashboards for farmers, buyers, and administrators.

The platform aims to: - Empower farmers with better market access and fair pricing - Connect buyers directly with local producers - Provide intelligent agricultural planning tools - Facilitate secure transactions and communication

Features

Core Functionality

- **User Authentication:** Secure registration and login with role-based access (Farmer, Buyer, Admin, Super Admin)
- **Crop Listings:** Farmers can list crops with photos, descriptions, pricing, and location data
- **Marketplace:** Buyers can browse and search crops by various criteria
- **Real-time Messaging:** Direct communication between farmers and buyers
- **Order Management:** Bulk ordering system with status tracking
- **Admin Panel:** User management, order approval, and system oversight

AI-Powered Features

- **AgroPlan AI:** Intelligent crop planning assistant using OpenAI GPT-4

- **Soil Analysis:** AI-driven recommendations based on soil type, pH, moisture, and location
- **Weather Integration:** Location-based weather forecasts and recommendations
- **Sustainability Insights:** Regenerative agriculture practices and environmental recommendations

Technical Features

- **Responsive Design:** Mobile-first approach with modern UI components
- **File Uploads:** Image storage for crop listings and soil analysis
- **Local Storage:** SQLite database with file-based image storage
- **Demo Mode:** Pre-configured scenarios for testing and demonstration
- **End-to-End Testing:** Playwright-based test suite

Technology Stack

Frontend

- **React 18:** Modern React with hooks and functional components
- **TypeScript:** Type-safe development
- **Vite:** Fast build tool and development server
- **shadcn/ui:** High-quality UI component library
- **Tailwind CSS:** Utility-first CSS framework
- **React Router:** Client-side routing
- **React Query:** Data fetching and state management
- **Lucide React:** Icon library

Backend

- **Node.js:** JavaScript runtime
- **Express.js:** Web framework for API development
- **SQLite:** Lightweight database with better-sqlite3
- **bcryptjs:** Password hashing
- **Multer:** File upload handling
- **CORS:** Cross-origin resource sharing
- **OpenAI API:** AI-powered agricultural recommendations

Development Tools

- **ESLint:** Code linting
- **Playwright:** End-to-end testing
- **Vite Plugin React:** React integration for Vite
- **TypeScript ESLint:** TypeScript linting

Deployment

- **Local Development:** Concurrent frontend/backend development servers

- **Production Ready:** Environment variable configuration
- **File Storage:** Local file system for images (easily replaceable with cloud storage)

Installation

Prerequisites

- Node.js (v16 or higher)
- npm or bun package manager
- OpenAI API key (for AI features)

Setup Steps

1. Clone the repository

```
git clone <repository-url>
cd agriconnect
```

2. Install dependencies

```
npm install
```

3. Environment configuration

```
cp .env.example .env
```

Edit `.env` with your configuration:

```
# OpenAI API Key (required for AI features)
OPENAI_API_KEY=your_openai_api_key_here
```

```
# Super Admin Credentials (optional)
SUPER_ADMIN_EMAIL=admin@example.com
SUPER_ADMIN_PASSWORD=secure_password
```

```
# Environment
NODE_ENV=development
```

4. Database setup

The SQLite database is automatically created and migrated when the server starts.

5. Start development servers

```
npm run dev:all
```

This runs both frontend (<http://localhost:5173>) and backend (<http://localhost:3001>) concurrently.

6. Access the application

- Frontend: <http://localhost:5173>
- Backend API: <http://localhost:3001>

Usage

User Roles

Farmers

- Register as a farmer
- Create crop listings with photos and details
- Manage existing listings (edit, delete, update status)
- View and respond to buyer inquiries
- Access AgroPlan AI for crop planning
- Track orders and earnings

Buyers

- Register as a buyer
- Browse crop listings with search and filters
- Place bulk orders
- Communicate directly with farmers
- Track order status

Administrators

- Manage all users (create, edit, delete)
- Approve pending orders
- View system analytics
- Access admin messaging

Key Workflows

Crop Listing Process

1. Farmer logs in and navigates to dashboard
2. Clicks “Add Crop” with details and photo upload
3. Crop appears in marketplace for buyers

Order Process

1. Buyer browses crops and adds to cart
2. Places order with contact information
3. Admin approves the order
4. Farmer receives notification and fulfills order

AI Planning

1. User provides farm data (detailed or simple input)
2. AI analyzes soil, weather, and crop history
3. Receives personalized recommendations

API Documentation

The API follows RESTful conventions and uses JSON for data exchange.

Base URL

`http://localhost:3001/api`

Authentication

Most endpoints require authentication via Bearer token in Authorization header:

Authorization: Bearer <user_id>

Endpoints

Authentication

- POST /auth/register - User registration
- POST /auth/login - User login

Crops

- GET /crops - List crops (with filtering)
- POST /crops - Create crop (multipart/form-data)
- PUT /crops/:id - Update crop
- DELETE /crops/:id - Delete crop

Users

- GET /users/:id - Get user profile

Orders

- POST /orders - Create order
- GET /orders - List orders
- PUT /orders/:id/approve - Approve order (admin only)

Messaging

- GET /messages - Get user messages
- POST /messages - Send message
- PUT /messages/:id/read - Mark message as read

Admin

- GET /admin/users - List all users (admin only)
- POST /admin/users - Create user (admin only)
- PUT /admin/users/:id - Update user (admin only)
- DELETE /admin/users/:id - Delete user (admin only)

AgroPlan AI

- POST /agroplan/analyze - Generate AI analysis (multipart/form-data)

Public

- GET /admins - Get list of administrators

Request/Response Examples

Create Crop

```
const formData = new FormData();
formData.append('farmerId', 'user-123');
formData.append('name', 'Maize');
formData.append('description', 'High-quality maize');
formData.append('quantity', '100');
formData.append('unit', 'kg');
formData.append('pricePerUnit', '50');
formData.append('image', file);

fetch('/api/crops', {
  method: 'POST',
  body: formData
});
```

AI Analysis

```
const formData = new FormData();
formData.append('inputMode', 'detailed');
formData.append('location', 'Nakuru, Kenya');
formData.append('soilType', 'loamy');
formData.append('soilPh', '6.8');

fetch('/api/agroplan/analyze', {
  method: 'POST',
  body: formData
});
```

Database Schema

The application uses SQLite with the following tables:

users

```
CREATE TABLE users (
  id TEXT PRIMARY KEY,
  email TEXT UNIQUE NOT NULL,
```

```

password_hash TEXT NOT NULL,
full_name TEXT,
user_type TEXT NOT NULL CHECK (user_type IN ('farmer', 'buyer', 'admin', 'super_admin')),
created_at TEXT DEFAULT CURRENT_TIMESTAMP
);

```

profiles

```

CREATE TABLE profiles (
  id TEXT PRIMARY KEY,
  avatar_url TEXT,
  location TEXT,
  phone TEXT,
  rating REAL DEFAULT 0,
  FOREIGN KEY (id) REFERENCES users(id) ON DELETE CASCADE
);

```

crops

```

CREATE TABLE crops (
  id TEXT PRIMARY KEY,
  farmer_id TEXT NOT NULL,
  name TEXT NOT NULL,
  description TEXT,
  quantity REAL NOT NULL,
  unit TEXT NOT NULL,
  price_per_unit REAL NOT NULL,
  harvest_date TEXT,
  location TEXT,
  image_url TEXT,
  status TEXT DEFAULT 'available' CHECK (status IN ('available', 'reserved', 'sold')),
  created_at TEXT DEFAULT CURRENT_TIMESTAMP,
  updated_at TEXT DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (farmer_id) REFERENCES users(id) ON DELETE CASCADE
);

```

messages

```

CREATE TABLE messages (
  id TEXT PRIMARY KEY,
  sender_id TEXT NOT NULL,
  receiver_id TEXT NOT NULL,
  content TEXT NOT NULL,
  read INTEGER DEFAULT 0,
  created_at TEXT DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (sender_id) REFERENCES users(id) ON DELETE CASCADE,

```

```
    FOREIGN KEY (receiver_id) REFERENCES users(id) ON DELETE CASCADE
);
```

orders

```
CREATE TABLE orders (
  id TEXT PRIMARY KEY,
  crop_id TEXT NOT NULL,
  buyer_id TEXT NOT NULL,
  quantity REAL NOT NULL,
  total_price REAL NOT NULL,
  buyer_contact TEXT,
  status TEXT DEFAULT 'pending' CHECK (status IN ('pending', 'confirmed', 'delivered', 'cancelled')),
  delivery_date TEXT,
  created_at TEXT DEFAULT CURRENT_TIMESTAMP,
  updated_at TEXT DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (crop_id) REFERENCES crops(id) ON DELETE CASCADE,
  FOREIGN KEY (buyer_id) REFERENCES users(id) ON DELETE CASCADE
);
```

agroplan_data

```
CREATE TABLE agroplan_data (
  id TEXT PRIMARY KEY,
  user_id TEXT NOT NULL,
  soil_type TEXT,
  location TEXT,
  previous_crops TEXT,
  recommendations TEXT,
  sustainability_score INTEGER,
  created_at TEXT DEFAULT CURRENT_TIMESTAMP,
  updated_at TEXT DEFAULT CURRENT_TIMESTAMP,
  FOREIGN KEY (user_id) REFERENCES users(id) ON DELETE CASCADE
);
```

Testing

The project includes end-to-end tests using Playwright.

Running Tests

1. Install Playwright browsers

```
npx playwright install
```

2. Run tests

```
npm run playwright:test
```


3. Run with UI

```
npx playwright test --ui
```

Test Coverage

The current test suite covers: - Add to cart functionality - Order placement workflow - Mocked API responses for crops and orders

Tests are designed to run without a live backend by intercepting API calls and providing mock responses.

Integration Testing

For full integration tests against the real backend:

```
npm run e2e:integration
```

This runs a PowerShell script that sets up the environment and runs comprehensive tests.

Contributing

1. Fork the repository
2. Create a feature branch (`git checkout -b feature/amazing-feature`)
3. Commit your changes (`git commit -m 'Add amazing feature'`)
4. Push to the branch (`git push origin feature/amazing-feature`)
5. Open a Pull Request

Development Guidelines

- Follow TypeScript best practices
- Use ESLint configuration
- Write meaningful commit messages
- Add tests for new features
- Update documentation as needed

Code Style

- Use functional components with hooks
- Follow React and TypeScript naming conventions
- Use Tailwind CSS classes for styling
- Keep components modular and reusable

License

This project is licensed under the MIT License - see the LICENSE file for details.

For more information or support, please contact the development team or open an issue on the project repository.